Lesson notes · for you

Tools are functions

AI agents · Upper secondary · Lesson 2

Time	Materials
20 min	Your phone or a school computer, for the code steps; Paper, a pen and a few small cards

Learning goals

- Write a tool as a Python function.
- Send each call to the right tool with a dictionary.
- Return errors instead of crashing or guessing.

Activity: Be the tools

1. Warm-up (3 min): Read Mr Okafor's story: the model asked for a bus tool that didn't exist. List three things that could happen next, and pick the best.
2. Cards (7 min): In a group of four, each person is one tool (weather, timetable, library, route). Write the details your tool needs on a card, and your results for two or three made-up cases.
3. Calls (6 min): One person reads calls aloud, some good, some with a missing detail, one for a tool nobody has. The right tool answers from its card; if something is wrong, it answers with an error instead.
4. Code (4 min): In the code step, call run with a tool that isn't in TOOLS, before and after your fix.

Use made-up villages, classes and books only, such as Kafi, Dala and 6B. The card game needs no devices.

Discussion

- Why is a guess worse than an error?
- How would you add a new tool to TOOLS?
- Which details would you check inside each tool?

Check yourself

- Write one call that should return an error, and the error it returns.

Try it this week
Invent another tool for your agent, such as a library search. Write its details, its result and two errors it should return, then add it to TOOLS in the code step and test it.

Worksheet

Tools are functions

AI agents · Upper secondary · Lesson 2

Name _____ Class _____ Date _____

1 **The model calls weather but leaves out the place. What should the weather tool return?**

◯ Kafi's weather, as a good guess ◯ An error: the place is missing ◯ An empty result

2 **Match each piece of your agent to its job.**

weather(args)	• • • •	Sends a call to its tool, or returns an error
TOOLS	• • • •	Turns the result into text for the model
run(name, args)	• • • •	Looks up the sky for a place
json.dumps(result)	• • • •	A dictionary from tool names to functions

3 **Finish run(name, args): if the tool isn't in TOOLS, return {"error": "unknown tool"}; otherwise run the tool with its details and return its result.**

```
SKY = {"Kafi": "sunny", "Dala": "rain"}

def weather(args):
    place = args.get("place")
    if place is None:
        return {"error": "missing place"}
    if place not in SKY:
        return {"error": "unknown place"}
    return {"sky": SKY[place]}

TOOLS = {"weather": weather}

def run(name, args):
    return TOOLS[name](args)
```

Write your program:

4 **Fill the gaps: run the call's tool if it exists, and turn the result into text for the model.**

```
name, args = parse_call(call)
if name _____ TOOLS:
    result = _____[name](args)
else:
    result = {"error": "unknown tool"}
text = json._____(result)
```

Word bank: `loads` `dumps` `TOOLS` `in` `args` `not in`

5 **If the model leaves out the place, the weather tool should return an error rather than guess the nearest village.**

◯ True ◯ False

Think of a tool you would give an agent at your school. What details would it need, and which errors should it return?

Check your answers

Tools are functions

AI agents · Upper secondary · Lesson 2

1 **An error: the place is missing**

A tool shouldn't guess a detail nobody gave.

2 **weather(args) — Looks up the sky for a place**
TOOLS — A dictionary from tool names to functions
run(name, args) — Sends a call to its tool, or returns an error
json.dumps(result) — Turns the result into text for the model

Each tool is a function; TOOLS maps names to those functions; run uses TOOLS, with a check, to reach the right one; json.dumps turns the result back into text the model can read.

3

```
SKY = {"Kafi": "sunny", "Dala": "rain"}

def weather(args):
    place = args.get("place")
    if place is None:
        return {"error": "missing place"}
    if place not in SKY:
        return {"error": "unknown place"}
    return {"sky": SKY[place]}

TOOLS = {"weather": weather}

def run(name, args):
    if name not in TOOLS:
        return {"error": "unknown tool"}
    return TOOLS[name](args)
```

Checking the name first turns a made-up tool into an error the model can read; TOOLS[name](args) then runs a real one.

4

```
name, args = parse_call(call)
if name in TOOLS:
    result = TOOLS[name](args)
else:
    result = {"error": "unknown tool"}
text = json.dumps(result)
```

Run the tool only if its name is in TOOLS; TOOLS[name] is the function.

5 **True**

A tool shouldn't guess a detail nobody gave: it would answer about the wrong village.